

Anomaly Detection in Dynamic & Graph-based Data

Dr. Charlie Abela

Department of Artificial Intelligence

University of Malta

Objectives



L-Università ta' Malta
Faculty of Information &
Communication Technology

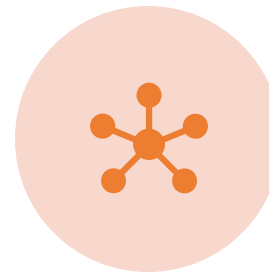
Department
of Artificial
Intelligence



CHARACTERISTICS OF
DYNAMIC DATA



TIME SERIES
ANOMALY DETECTION



GRAPH-BASED
ANOMALY DETECTION



REAL-WORLD
USE CASE



Anomaly Detection on Dynamic Data

Understanding Time Series Data



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Trend:** Long-term movement in the series (increasing, decreasing, or stable)
- **Seasonality:** Regular and predictable patterns that repeat over known periods
- **Cyclicity:** Irregular fluctuations without fixed frequency
- **Noise:** Random variation that cannot be explained by the model
- **Autocorrelation:** Correlation between observations as a function of time lag

Types of Time Series Anomalies

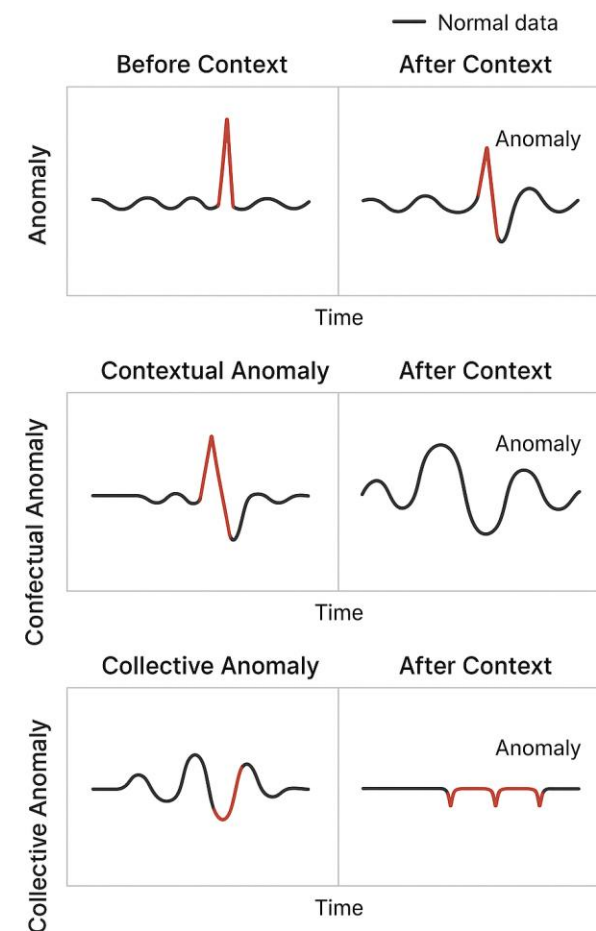


- **Point Anomalies:** Individual values deviating significantly from the expected range
 - *Example:* Server CPU spike to 99% when normal operation is 10-30%
- **Contextual Anomalies:** Values normal in one context but anomalous in another
 - *Example:* 100 logins per minute during business hours (normal) vs. same activity at 3 AM (anomalous)
- **Collective Anomalies:** Sequences that are unusual as a whole
 - *Example:* Normal heart rate values that form an abnormal rhythm when viewed together

Types of Time Series Anomalies



- This visualization illustrates three fundamental types of anomalies in time series data, demonstrating how context dramatically alters anomaly interpretation.
- The top row shows a point anomaly, where an obvious spike remains anomalous with additional context, though revealing it's part of a more complex pattern with both peak and valley components.
- The middle row demonstrates a contextual anomaly—a pattern initially appearing abnormal when viewed in isolation is revealed to be perfectly normal when seen within a broader cyclical context.
- The bottom row presents a collective anomaly, where what seems like a single anomalous dip represents multiple consecutive dips forming a distinct pattern that only becomes apparent when viewed as a collective sequence.



Statistical approaches: Moving Window



- **Methodology:**

- Define sliding window of size w over time series $\{x_1, x_2, \dots, x_T\}$
- For each point x_t , calculate statistics over previous window $[t - w, t - 1]$
- Flag anomalies based on deviation from these statistics

- **Mathematical Formulation:**

- Window mean: $\mu_t = \frac{1}{w} \sum_{i=t-w}^{t-1} x_i$
- Window standard deviation: $\sigma_t = \sqrt{\frac{1}{w} \sum_{i=t-w}^{t-1} (x_i - \mu_t)^2}$
- Anomaly if: $|x_t - \mu_t| > k \cdot \sigma_t$ where k is sensitivity parameter

Statistical approaches: Moving Window



- **Example:**

- Time series:

- Window $w = 5$, sensitivity $k = 3$
 - For $t = 6$ (value 30):
 - Window:

$$\mu_6 = \frac{10+12+11+13+10}{5} = 11.2$$

$$\sigma_6 = \sqrt{\frac{(10-11.2)^2+(12-11.2)^2+(11-11.2)^2+(13-11.2)^2+(10-11.2)^2}{5}} = 1.16$$

Threshold: $\mu_6 \pm k\sigma_6 = 11.2 \pm 3 \times 1.16 = [7.72, 14.68]$

Since $30 > 14.68$, point is flagged as anomaly

Forecast-Based Approaches



• Conceptual Framework:

- Train model on historical normal data to learn expected patterns
- Generate forecasts with prediction intervals
- Identify anomalies as observations significantly deviating from forecasts

• Mathematical Formulation:

- For time series model f , predict value at time t : $\hat{y}_t = f(y_{t-1}, y_{t-2}, \dots, y_{t-p})$
- Compute residual: $r_t = y_t - \hat{y}_t$
- Estimate prediction interval: $PI_t = \hat{y}_t \pm z_{\alpha/2} \cdot \sigma_r$
- Anomaly if: $y_t \notin PI_t$ or equivalently $|r_t| > z_{\alpha/2} \cdot \sigma_r$

Forecast-Based Approaches: Example



- Model predicts store traffic at 2 PM to be 120 ± 15 customers
- Actual count: 162 customers
- Standardized residual: $\frac{162-120}{15} = 2.8$
- Using 99% confidence ($z_{0.005} = 2.576$: $(2.8 > 2.576$,so flag as anomaly

Why Neural Networks for Time Series?



L-Università ta' Malta
Faculty of Information &
Communication Technology

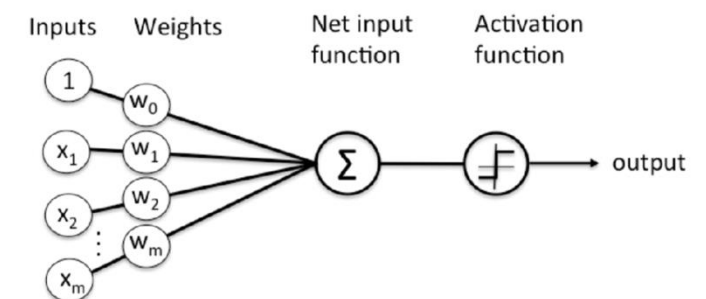
Department
of Artificial
Intelligence

- **Limitations of Classical Methods:**

- **Statistical Methods:** Assume specific distribution patterns; struggle with non-linearity
- **ARIMA Models:** Limited memory; primarily linear relationships
- **Exponential Smoothing:** Cannot capture complex interactions between components

- **Neural Network Advantages:**

- **Non-linearity:** Capture complex, non-linear patterns using activation functions
- **Automatic Feature Learning:** Discover relevant features without manual engineering
- **Flexibility:** Adapt to various data types and domains
- **Scalability:** Handle high-dimensional, large-scale time series



Feed-Forward Neural Network



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Architectural Overview:**

- **Input Layer:** Receives feature vector

$$\mathbf{x} \in \mathbb{R}^n$$

- **Hidden Layers:** Apply transformations

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

- **Output Layer:** Produces prediction

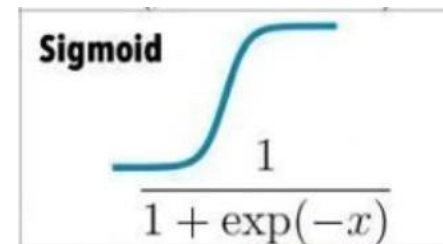
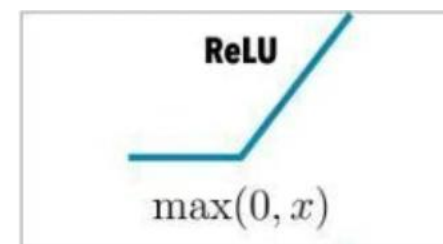
$$\mathbf{y} = g(V\mathbf{h} + \mathbf{c})$$

- **Mathematical Foundation:**

- Each neuron computes:

$$h_j = \sigma \left(\sum_{i=1}^n w_{ji} x_i + b_j \right)$$

- Where σ is activation function (e.g.,
ReLU: $\sigma(z) = \max(0, z)$)



Feed-Forward Neural Network

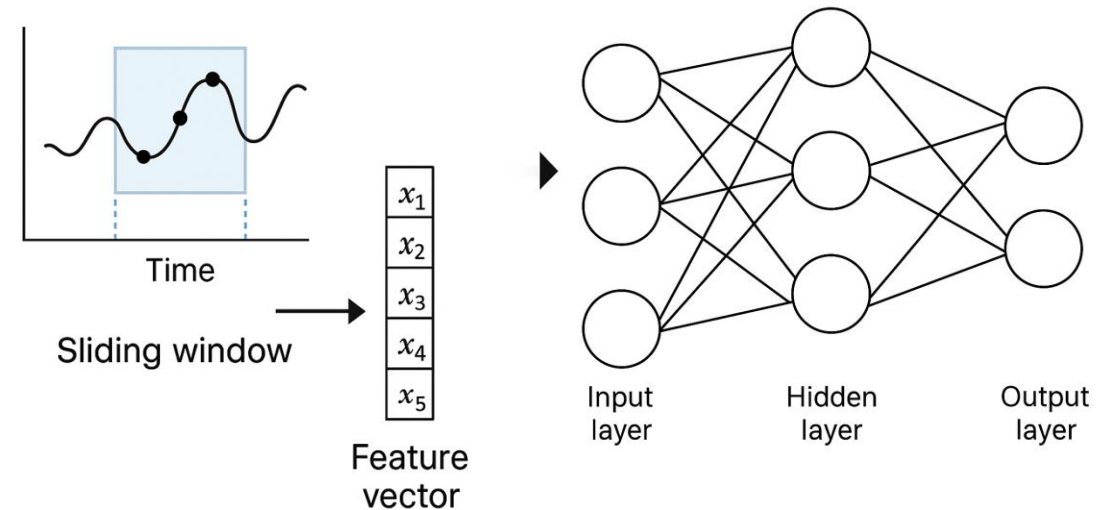


- **For Time Series:**

- Convert window of time steps to feature vector
- E.g., predict x_t using $[x_{t-w}, x_{t-w+1}, \dots, x_{t-1}]$

- **Fundamental Limitation:**

- Fixed input size; cannot handle variable-length sequences
- No weight sharing across time steps; inefficient for temporal patterns
- No memory of states beyond the input window



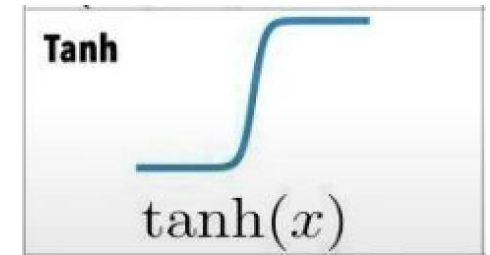
Enter Recurrent Neural Networks



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Key Innovation:** Internal memory allows information persistence across time steps
- **RNN Cell Mechanics:**
 - At each time step t , processes input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$
 - Updates hidden state: $\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b}_h)$
 - Produces output: $\mathbf{y}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y$
- **Key Insight:** Same weights used across all time steps (parameter sharing)
- **Benefits:**
 - Handles variable-length sequences
 - Maintains state information between steps
 - Efficient parameter usage through weight sharing



RNN example and walkthrough



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- Simple univariate time series prediction with minimal RNN

- **Setup:**

- Input sequence: [1.0,2.0,3.0]
- Initial hidden state: $h_0 = 0$
- Weight matrices (simplified):
 $W_{xh} = 0.8$, $W_{hh} = 0.5$, $W_{hy} = 1.0$
- Biases: $b_h = 0$, $b_y = 0$

Observations:

Hidden state captures history: h_3 contains information from all previous inputs With same weights, RNN learns to predict next value in sequence

Step-by-Step Calculation:

Step 1:

$$\begin{aligned}h_1 &= \tanh(W_{hh} \cdot h_0 + W_{xh} \cdot x_1 + b_h) \\&= \tanh(0.5 \cdot 0 + 0.8 \cdot 1.0 + 0) = \tanh(0.8) \approx 0.66 \\y_1 &= W_{hy} \cdot h_1 + b_y = 1.0 \cdot 0.66 + 0 = 0.66\end{aligned}$$

Step 2:

$$\begin{aligned}h_2 &= \tanh(W_{hh} \cdot h_1 + W_{xh} \cdot x_2 + b_h) \\&= \tanh(0.5 \cdot 0.66 + 0.8 \cdot 2.0 + 0) = \tanh(1.93) \\&\approx 0.96 \\y_2 &= W_{hy} \cdot h_2 + b_y = 1.0 \cdot 0.96 + 0 = 0.96\end{aligned}$$

Step 3:

$$\begin{aligned}h_3 &= \tanh(W_{hh} \cdot h_2 + W_{xh} \cdot x_3 + b_h) \\&= \tanh(0.5 \cdot 0.96 + 0.8 \cdot 3.0 + 0) = \tanh(2.88) \\&\approx 0.99 \\y_3 &= W_{hy} \cdot h_3 + b_y = 1.0 \cdot 0.99 + 0 = 0.99\end{aligned}$$

Vanishing Gradient Problem



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- When training RNNs, gradients diminish exponentially as they flow backward through time steps, preventing the network from learning long-term dependencies.
- Why It Happens
 - **Backpropagation Through Time:** Gradients must flow backward through all time steps
 - **Multiplicative Chain Rule:** Gradient at time step $t-n$ requires multiplication of n gradient terms:
$$\frac{\partial L}{\partial W} \propto \prod_{i=1}^n \frac{\partial h_i}{\partial h_{i-1}}$$
 - **Small Derivatives:** Activation functions (tanh/sigmoid) have derivatives ≤ 0.25
 - **Compounding Effect:** If each gradient term is 0.1, after just 10 steps: $0.1^{10} = 10^{-10}$ (effectively zero)
- Consequences
 - **Short-term Memory:** Network can only learn dependencies across few time steps
 - **Early Time Steps Stagnate:** Remote inputs have negligible effect on output
 - **Training Difficulty:** Network convergence slows dramatically or fails

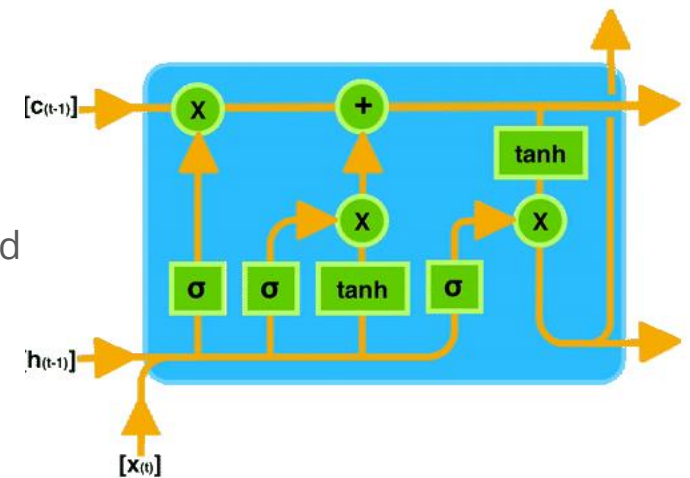
Long Short-Term Memory (LSTM) Networks: The Solution



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- Designed specifically to overcome vanishing gradients
- Introduces “cell state” as long-term memory highway
- Controlled information flow through specialized gate mechanisms
- **Key Components and Their Roles:**
 - **Cell State** (C_t) :Long-term memory storage; gradient highway with uninterrupted
 - **Hidden State** (h_t) :Short-term memory and output to next layer
 - **Forget Gate** (f_t) :Controls what information to discard from cell state
 - **Input Gate** (i_t) :Controls what new information to add to cell state
 - **Output Gate** (o_t) :Controls what information from cell state to output
- **Historical Context:**
 - Developed by Hochreiter & Schmidhuber in 1997 to solve vanishing gradient problem
 - Enabled breakthrough applications in speech recognition, machine translation, etc.



<http://tiny.cc/556k001>

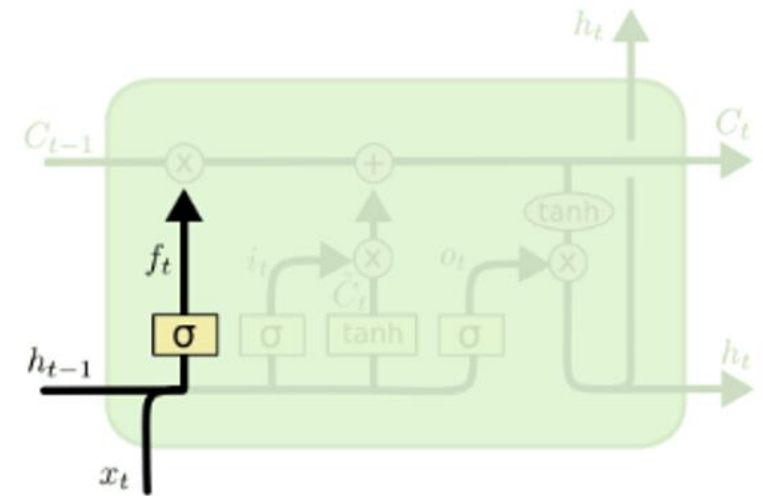
Inside the LSTM Cell: Gate Mechanisms



• Forget Gate - Deciding What to Discard:

- Function: Controls what information from previous cell state to keep or discard
- Formula: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- Output range: for each cell state element (0 = forget completely, 1 = keep completely)
- Example: In language model, may forget gender information when crossing sentence boundary
- Weight dimensions:

$$W_f \text{ is } h_{dim} \times (h_{dim} + x_{dim}), b_f \text{ is } h_{dim} \times 1$$



Inside the LSTM Cell: Gate Mechanisms

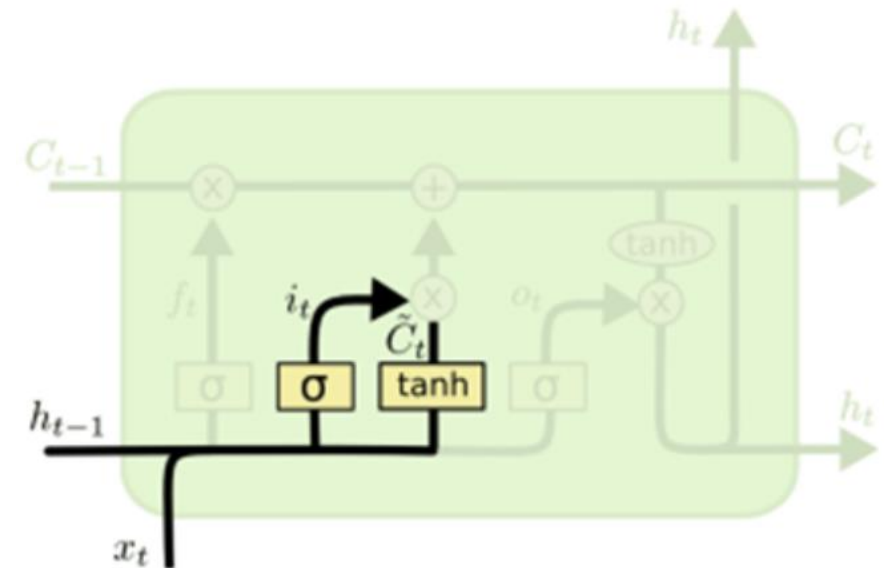


• Input Gate - Deciding What to Update:

- Function: Controls which new information will be added to cell state
- Formula: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- Candidate values:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- Output range: for gate, $[-1, 1]$ for candidate values
- Example: In sequence anomaly detection, may emphasize unusual patterns

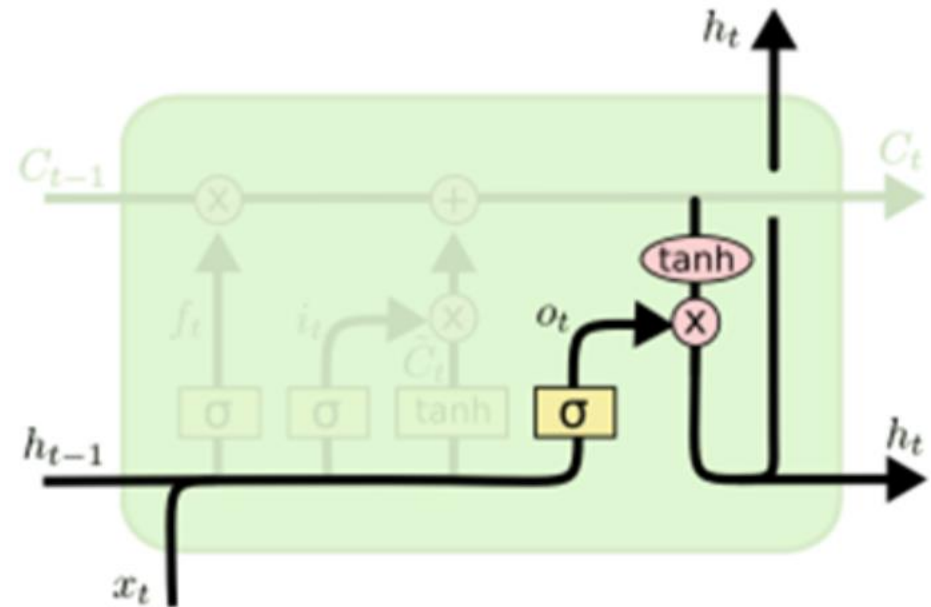


Inside the LSTM Cell: Gate Mechanisms



- **Output Gate - Deciding What to Output:**

- Function: Controls what parts of cell state to expose as output
- Formula: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
- Output range: for each output element
- Example: May output information relevant to current prediction but keep another context hidden



LSTM Cell State: Memory Highway



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- The cell state (C_t) serves as a "conveyor belt" allowing information to flow unchanged through the LSTM network, solving the vanishing gradient problem.

$$C_t = f_t \odot C_{(t-1)} + i_t \odot \hat{C}_t$$

- The gates control the information flow:
 - Forget gate: Decides what to erase from previous memory
 - Values near 0: "forget this"; Values near 1: "keep this"
 - Input Gate: Controls what new information gets stored
 - Filters candidate values ($\hat{C}_t$) before adding to cell state
 - Output Gate: Determines what parts of cell state to expose
 - Hidden state (h_t) = $o_t \odot \tanh(C_t)$
- It works because:
 - Uninterrupted gradient flow: direct path for backpropagation
 - Selective memory: can maintain important information for hundreds of steps
 - Controlled updates: gates learn which information is worth remembering

LSTM Step-by-Step Walkthrough (Part 1)



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Example Scenario:** Processing a point in financial time series
- **Input Values:**
 - Current input: $x_t = [0.5]$ e.g., normalized stock price change)
 - Previous hidden state: $h_{t-1} = [0.2]$ information from past)
 - Previous cell state: $C_{t-1} = [0.3]$ long-term memory)
- **Simplified Weights (For Demonstration):**
 - $W_f = [0.7, 0.2]$, $b_f = [-0.3]$ forget gate parameters)
 - $W_i = [0.4, 0.2]$, $b_i = [0.0]$ input gate parameters)
 - $W_c = [0.3, 0.4]$, $b_c = [0.0]$ candidate value parameters)

LSTM Step-by-Step Walkthrough (Part 1)



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

• Gate Calculations:

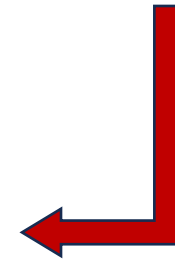
Forget gate:

$$\begin{aligned}f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\f_t &= \sigma([0.7, 0.2] \cdot [0.2, 0.5] + [-0.3]) \\f_t &= \sigma(0.7 \times 0.2 + 0.2 \times 0.5 - 0.3) \\f_t &= \sigma(0.14 + 0.1 - 0.3) = \sigma(-0.06) = 0.485\end{aligned}$$



Input gate:

$$\begin{aligned}i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\i_t &= \sigma([0.4, 0.2] \cdot [0.2, 0.5] + [0.0]) \\i_t &= \sigma(0.4 \times 0.2 + 0.2 \times 0.5) = \sigma(0.18)\end{aligned}$$



Candidate values:

$$\begin{aligned}\tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \\\tilde{C}_t &= \tanh([0.3, 0.4] \cdot [0.2, 0.5] + [0.0]) \\\tilde{C}_t &= \tanh(0.3 \times 0.2 + 0.4 \times 0.5) = \tanh(0.26) = 0.255\end{aligned}$$

LSTM Step-by-Step Walkthrough (Part 2)



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

• Gate Calculations:

Cell State Update:

$$\begin{aligned}C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\C_t &= 0.485 \times 0.3 + 0.545 \times 0.255 \\C_t &= 0.1455 + 0.139 = 0.2845\end{aligned}$$

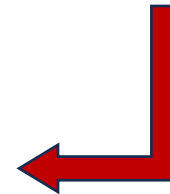


Output Gate Calculation:

$$\begin{aligned}o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ \text{Using } W_o &= [0.5, 0.3] \text{ and } b_o = [0.2]: \\ o_t &= \sigma([0.5, 0.3] \cdot [0.2, 0.5] + [0.2]) \\ o_t &= \sigma(0.5 \times 0.2 + 0.3 \times 0.5 + 0.2) \\ o_t &= \sigma(0.1 + 0.15 + 0.2) = \sigma(0.45) = 0.611\end{aligned}$$

Hidden State Update:

$$\begin{aligned}h_t &= o_t \odot \tanh(C_t) \\ h_t &= 0.611 \times \tanh(0.2845) \\ h_t &= 0.611 \times 0.277 = 0.169\end{aligned}$$





• Interpretation:

- Forget gate partially cleared old cell state (keeping ~49%)
- Input gate allowed selective addition of new information (~55% of candidate)
- Final hidden state is controlled exposure of cell state

• Information Flow Analysis:

- Previous cell state contribution: ~51% forgotten, ~49% retained
- New information contribution: ~55% of candidate values added
- Output exposure: ~61% of transformed cell state revealed in output

Why LSTMs Excel at Sequence Learning



- **Architectural Advantages:**

- Controlled Information Flow: Gates learn what's important to remember/forget
- Gradient Highway: Cell state provides uninterrupted gradient flow
- Selective Memory: Can remember important information for hundreds of steps
- Adaptability: Can adjust memory usage based on sequence requirements

- **Practical Capabilities:**

- Long-Range Dependencies: Connect events separated by many time steps
 - Example: Remember subject of sentence to ensure verb agreement
- Pattern Recognition: Identify complex temporal patterns
 - Example: Detect irregular heartbeats in ECG that require context
- Selective Attention: Focus on relevant parts of history
 - Example: In stock price anomaly detection, focus on similar historical patterns
- Context Integration: Combine historical context with current observations
 - Example: Flag unusual network traffic considering time of day and recent patterns

From LSTMs to Anomaly Detection: The Autoencoder Approach



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- An LSTM Autoencoder is like a smart photocopier for sequences: it learns to copy normal patterns so well that it struggles when given something unusual.
- **Autoencoder Fundamentals:**
 - 1.Encoder:** Reads your sequence and creates a "summary" (like compressing a file)
 - 2.Bottleneck:** The tiny summary contains only the essential patterns
 - 3.Decoder:** Tries to recreate the original sequence from just the summary
- **Anomaly Detection Principle:**
 - Train it on normal data only (like healthy heartbeats)
 - When it sees something weird (like an irregular heartbeat):
 - Normal pattern → Good recreation → Low error
 - Abnormal pattern → Poor recreation → High error → "ANOMALY ALERT!"

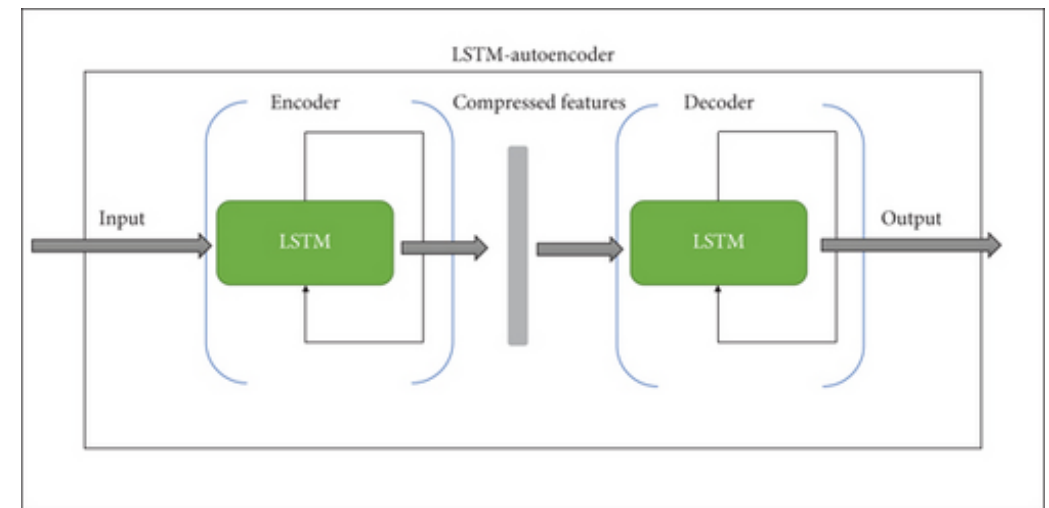
LSTM Autoencoder: Architecture



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Encoder:**
 - Series of LSTM layers that process input sequences
 - Learns to compress temporal information into a compact representation
 - Final hidden state captures essential patterns from the entire sequence
- **Bottleneck:**
 - Compact representation of the original sequence
 - Acts as the information bottleneck that forces learning of efficient encodings
 - Dimensionality significantly smaller than input sequence
 - Represents the "essence" of what makes normal sequences normal
- **Decoder:**
 - LSTM layers that mirror the encoder but work in reverse
 - Initialized using the encoder's final state/representation
 - Reconstructs the original sequence step by step
 - Often uses teacher forcing during training (providing correct previous output)
 - Final layer projects to match original input dimensions



<https://onlinelibrary.wiley.com/doi/epdf/10.1155/2022/3622426>

LSTM Autoencoder: Design Consideration



- **Bottleneck size:**
 - Larger bottleneck: More accurate reconstruction but may overfit to training data
 - Smaller bottleneck: Forces model to learn more efficient representations
 - Finding the right balance is critical: too small creates underfitting, too large defeats compression purpose
- **Layer depth:**
 - Deeper architectures (more layers): Can capture more complex patterns
 - Shallower architectures: Easier to train, may suffice for simpler sequences
 - Diminishing returns after certain depth; start simple and increase as needed
- **LSTM Variants:**
 - Bidirectional LSTMs: Capture both forward and backward dependencies
 - Stacked LSTMs: Allow for hierarchical feature extraction
 - Attention mechanisms: Can help focus on relevant parts of sequences

LSTM Autoencoder: Example (Part 1)



- **Scenario:** Server CPU Utilization Monitoring
- **Input Sequence (Normal Daily Pattern):**
 - $X = [10\%, 12\%, 11\%, 13\%, 12\%]$ (normalized to $[0.1, 0.12, 0.11, 0.13, 0.12]$)
- **LSTM Autoencoder Processing:**
 - **Encoder:** - Processes sequence through LSTM layers - Final hidden state becomes latent vector $\mathbf{z} = [0.3, 0.7]$ (simplification for illustration)
 - **Decoder:** - Initialized with $\mathbf{z}$ - Generates reconstruction sequence - Output: $\hat{X} = [0.098, 0.123, 0.108, 0.132, 0.119]$
- **Reconstruction Error Calculation:**
 - Mean Squared Error: $MSE = \frac{1}{5} \sum_{t=1}^5 (x_t - \hat{x}_t)^2$
 - $MSE = \frac{1}{5} [(0.1 - 0.098)^2 + (0.12 - 0.123)^2 + (0.11 - 0.108)^2 + (0.13 - 0.132)^2 + (0.12 - 0.119)^2]$
 - $MSE = \frac{1}{5} [0.000004 + 0.000009 + 0.000004 + 0.000004 + 0.000001]$
 - $MSE = \frac{0.000022}{5} = 0.0000044$ (very low error $\rightarrow$ normal pattern)

LSTM Autoencoder: Example (Part 2)



- **Input Sequence (Anomalous with Spike):**
 - $X_{anom} = [0.1, 0.12, 0.5, 0.13, 0.12]$ (unusual spike at $t=3$)
- **LSTM Autoencoder Processing:**
 - **Encoder:** - Processes anomalous sequence - Latent vector $\mathbf{z}_{anom} = [0.8, 0.4]$ (different due to anomaly)
 - **Decoder:** - Initialized with $\mathbf{z}_{anom}$ - Generates reconstruction based on learned normal patterns - Output: $\hat{X}_{anom} = [0.102, 0.125, 0.21, 0.135, 0.122]$ - Note: 0.21 is much lower than original 0.5 - model “smooths out” the spike
- **Reconstruction Error Calculation:**
 - $MSE = \frac{1}{5} [(0.1 - 0.102)^2 + (0.12 - 0.125)^2 + (0.5 - 0.21)^2 + (0.13 - 0.135)^2 + (0.12 - 0.122)^2]$
 - $MSE = \frac{1}{5} [0.000004 + 0.000025 + 0.0841 + 0.000025 + 0.000004]$
 - $MSE = \frac{0.084158}{5} = 0.0168$ (much higher error $\rightarrow$ anomaly)

LSTM Autoencoder: Example (Part 2 continued)



- **Threshold Decision:**

- Based on distribution of errors from validation set
- Example threshold ($\mu + 3\sigma$): $0.0000044 + 3 \times 0.00001 = 0.0000344$
- Since $0.0168 \gg 0.0000344$, confidently flag as anomaly

- **Interpretation:**

- Model learned normal CPU patterns, expects smooth transitions
- Cannot reconstruct sudden spike because it contradicts learned patterns
- High reconstruction error precisely at the anomalous time step

Anomaly Detection Process with LSTM Autoencoder



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **End-to-End Process Flow:**
 - **1. Data Preparation:**
 - **Cleaning:** Handle missing values, remove outliers from training data
 - **Normalization:** Scale features to similar ranges (e.g., 0-1 or z-score)
 - **Sequence Generation:** Create overlapping windows from time series
 - Formula: For series $\{x_1, x_2, \dots, x_N\}$ and window length L
 - Create sequences $\{[x_1, \dots, x_L], [x_2, \dots, x_{L+1}], \dots, [x_{N-L+1}, \dots, x_N]\}$
 - **2. Training Phase:**
 - Train autoencoder on normal sequences only
 - Monitor validation loss to prevent overfitting
 - Save model weights at lowest validation loss

Anomaly Detection Process with LSTM Autoencoder



- **End-to-End Process Flow (continued):**

- 3. Threshold Selection:**

- 1. Apply model to validation set of normal data
 - 2. Calculate error distribution statistics
 - 3. Set threshold based on desired sensitivity:
 - 1. Statistical: threshold = $\mu + k\sigma$ (e.g., $k = 3$ for 99.7% confidence)
 - 2. Percentile: threshold = percentile _{q} (errors) (e.g., $q = 99$)

- 4. Detection Phase:**

- 1. Preprocess new data identically to training data
 - 2. Generate reconstruction and error for each sequence
 - 3. Flag as anomaly if error > threshold

- 5. Post-Processing:**

- 1. Group contiguous anomalous sequences
 - 2. Filter short anomalies (noise reduction)
 - 3. Calculate anomaly severity scores

Case Study: LSTM-AE for ECG Anomaly Detection



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Problem Statement:**

- The case study tackles a significant healthcare challenge: automating the detection of cardiac arrhythmias that are typically identified through manual ECG review. The MIT-BIH Arrhythmia Database used is a gold standard dataset in cardiac signal processing research, containing:
 - 48 half-hour ECG recordings (representing ~24 hours of cardiac data)
 - Over 109,000 annotated beats
 - Multiple arrhythmia types (important for comprehensive evaluation)

- **Approach:** The methodology follows the core LSTM autoencoder principles but with healthcare-specific considerations:

- **Data Preparation:** 5-second segments at 100Hz sampling rate provide 500-point windows-enough to capture multiple heartbeat cycles while maintaining temporal pattern recognition
- **Semi-supervised Learning:** Training exclusively on normal sinus rhythm segments follows the theoretical principle we saw earlier-the model learns only "normal" patterns
- **Testing on Both Classes:** Evaluating on both normal and arrhythmic segments allows for robust performance assessment

Skarga-Bandurova, I., Biloborodova, T., Skarha-Bandurov, I., Boltov, Y. and Derkach, M., 2021. A Multilayer LSTM Auto-Encoder for Fetal ECG Anomaly Detection. In *pHealth 2021* (pp. 147-152). IOS press.

Case Study: LSTM-AE for ECG Anomaly Detection



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Model Architecture:** The model uses a progressive narrowing architecture:
 - Encoder with decreasing units (128→64→32) to progressively compress information
 - 16-dimensional bottleneck (significantly smaller than the 500 input points)
 - Symmetric decoder expanding back to original dimensions
 - Final Dense(1) layer to output the reconstructed signal
- **Results:** The results show clear differentiation between normal and abnormal patterns:
 - Normal reconstruction error ($\mu=0.037$, $\sigma=0.011$) vs. arrhythmic error ($\mu=0.218$, $\sigma=0.083$)
 - ~6× higher mean reconstruction error for arrhythmias
 - Strong performance metrics (ROC-AUC=0.967, Precision=0.93, Recall=0.91)
 - Particularly effective for ventricular arrhythmias (the most dangerous type)
- **Clinical Impact:**
 - 5x faster preliminary screening compared to manual review
 - 71% reduction in false alarms compared to rule-based systems
 - Localized anomaly regions within long recordings



Anomaly Detection on Graph-based Data

From Sequences to Networks



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Network Data Ubiquity:**

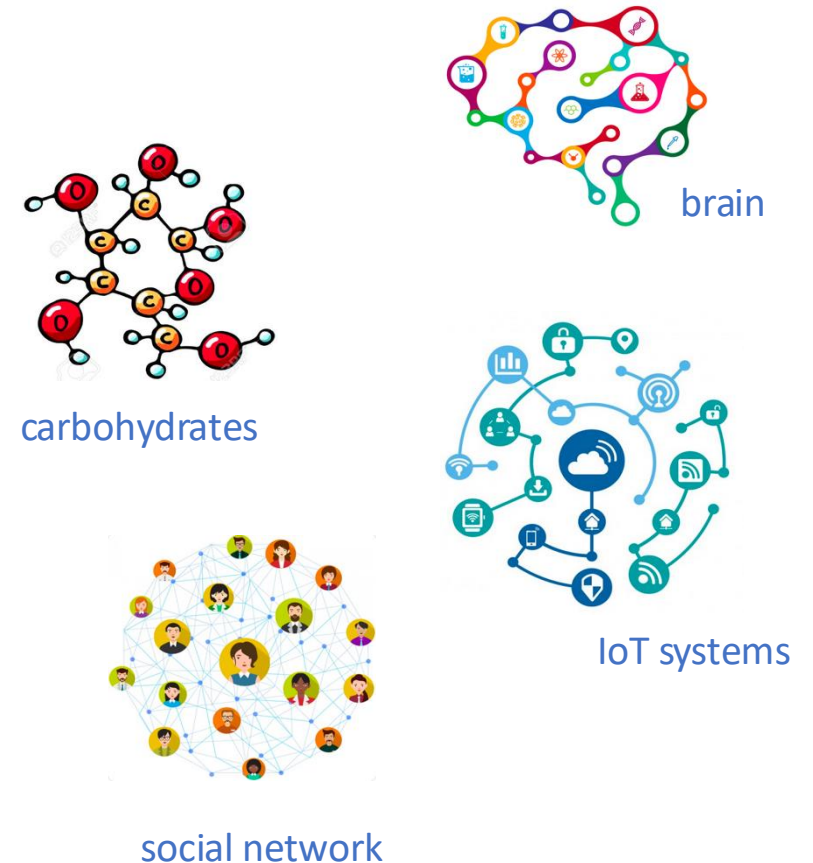
- Social networks: friendship, interactions
- Financial networks: transactions, trading relationships
- Computer networks: communication patterns
- Biological networks: protein interactions, neural connections

- **Why Graph Perspective Matters:**

- **Relational Information:** Entity behavior influenced by connections
- **Structural Patterns:** Anomalies often exist in relationship structure
- **Collective Behavior:** Groups of entities acting together abnormally

- **Motivating Example: Financial Fraud Detection**

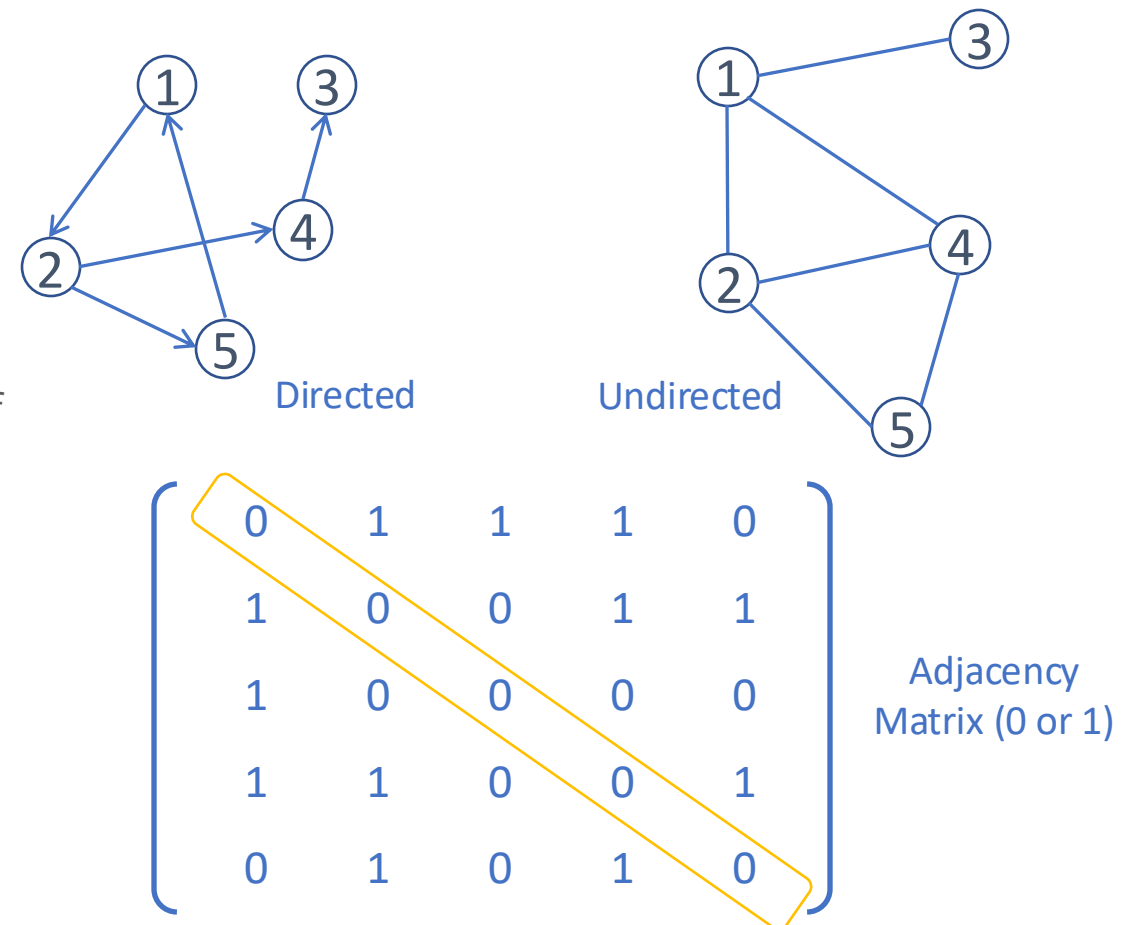
- **Non-Graph View:** Transaction of \$500 appears normal in isolation
- **Graph View:** Same transaction reveals suspicious pattern:
 - Part of a cycle where money returns to originator
 - Connected to recently created accounts with minimal history
 - Forms a bipartite structure typical of money laundering



Graph Fundamentals



- **Graph Definition:** $G = (V, E)$
 - $V = \{v_1, v_2, \dots, v_n\}$ is the set of nodes (vertices)
 - $E \subseteq V \times V$ is the set of edges
 - Directed edge: ordered pair (v_i, v_j)
 - Undirected edge: unordered pair $\{v_i, v_j\}$
- **Attributed Graph:** $G = (V, E, X_V, X_E)$
 - X_V is a matrix of node features where $X_V[i] = \text{features of node } v_i$
 - X_E is a matrix of edge features where $X_E[i, j] = \text{features of edge } (v_i, v_j)$
- **Graph Representations:**
 - **Adjacency Matrix:** $A \in \{0, 1\}^{n \times n}$ or $\mathbb{R}^{n \times n}$ for weighted
 - $A_{ij} = 1$ if edge exists between nodes i and j , else 0
 - For directed graphs: $A_{ij} = 1$ if edge from i to j
 - For weighted graphs: $A_{ij} = w_{ij}$ (edge weight)



Types of Graph Anomalies



- **Contextual Node Anomalies:** Nodes whose features are inconsistent with their neighborhood
 - Example: Luxury account in a cluster of budget accounts
 - Formal: Node v where X_v deviates from aggregated neighborhood features $\text{AGG}(\{X_u: u \in \mathcal{N}(v)\})$
- **Edge Anomalies:**
 - Unexpected Connection:** Edge connecting nodes that should not be related
 - Example: Transaction between entities in disparate communities
 - Formal: Edge (u, v) where probability $P(A_{uv} = 1 | X_u, X_v)$ is very low
 - Weight Anomalies:** Edge with abnormal weight given the connected nodes
 - Example: \$50,000 transfer between family members who typically exchange \$50
 - Formal: Edge (u, v) with weight w_{uv} where $|w_{uv} - \mathbb{E}[w_{uv} | X_u, X_v]| > \tau$
- **Subgraph Anomalies:**
 - Dense Subgraph:** Unusually dense interconnections within a group
 - Example: Fraud ring with all participants connected to each other
 - Formal: Subgraph $G_S = (V_S, E_S)$ where density
$$\frac{|E_S|}{\binom{|V_S|}{2}} \gg \frac{|E|}{|V|}$$
 - Structural Pattern:** Subgraphs with unusual topology
 - Example: Star pattern where central node controls many peripheral nodes
 - Formal: Subgraph with motif frequency significantly different from background

Traditional Graph Anomaly Detection Methods



- **Statistical Approaches:**

- **Degree-based Detection:**

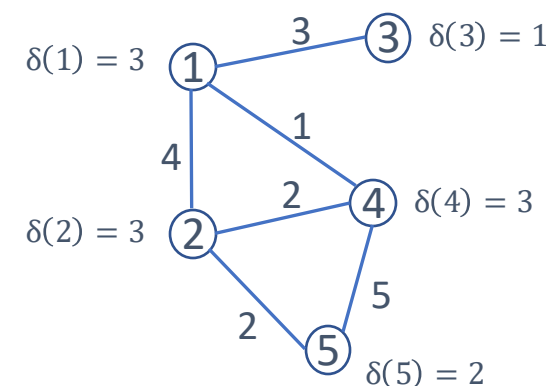
- Method: Identify nodes with statistically significant degree deviation
 - Formula: $\text{score}(v) = \frac{|d_v - \mu_d|}{\sigma_d}$ where d_v is degree of node v
 - Threshold: Flag if score > k (typically 3)

- **Centrality Measures:**

- Betweenness Centrality: $BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$ where σ_{st} is # of shortest paths from s to t , and $\sigma_{st}(v)$ is # passing through v
 - Flags nodes with unusually high influence over information flow

- **Clustering Coefficient:**

- Formula: $C_v = \frac{2|\{e_{jk}: v_j, v_k \in \mathcal{N}(v), e_{jk} \in E\}|}{|\mathcal{N}(v)|(|\mathcal{N}(v)|-1)}$
 - Detects nodes whose neighborhoods are unusually sparse/dense



0	4	3	1	5
3	0	6	2	2
3	6	0	4	8
1	2	4	0	4
5	2	8	4	0

Shortest paths

Traditional Graph Anomaly Detection Methods



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

• **Challenges & Limitations:**

- Only captures simple structural properties
- Treats each metric independently
- Cannot detect complex patterns
- No incorporation of node/edge attributes
- Requires domain knowledge for threshold setting

Algorithmic Example: Z-score on Node Degrees



```
import networkx as nx
import numpy as np

# Load graph
G = nx.Graph()
# Add nodes and edges...

# Calculate degrees
degrees = [d for n, d in G.degree()]
mean_degree = np.mean(degrees)
std_degree = np.std(degrees)

# Calculate z-scores
z_scores = [(n, (d - mean_degree) / std_degree) for n, d in G.degree()]

# Find anomalies
anomalies = [n for n, z in z_scores if abs(z) > 3]
```

Graph Embeddings



- **Core Concept:**

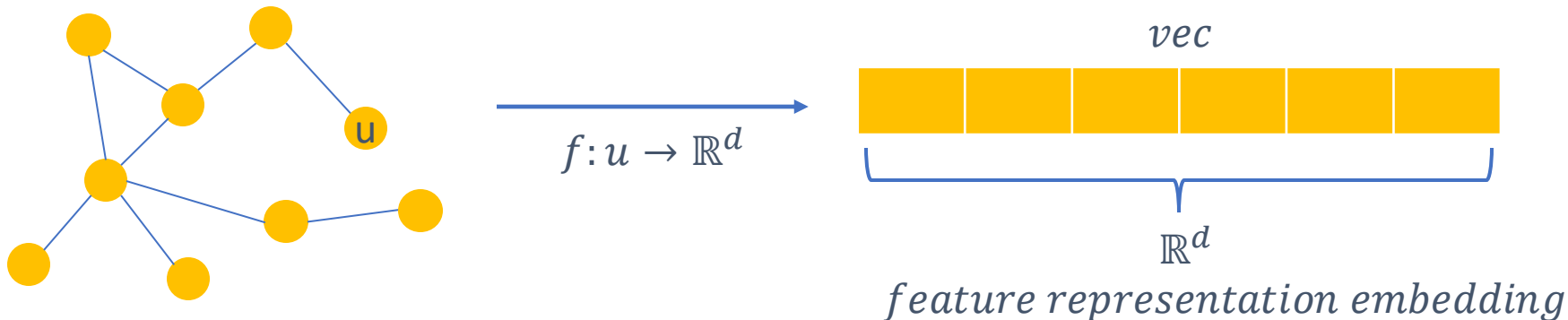
- Transform discrete graph elements into continuous vector space
- Encode structural information into dense, low-dimensional vectors
- Enable application of traditional ML algorithms to graph data

- **Formal Definition:**

- For graph $G = (V, E)$, an embedding is a mapping $f: V \rightarrow \mathbb{R}^d$
- Each node $v \in V$ is represented by vector $\mathbf{z}_v \in \mathbb{R}^d$ where $d \ll |V|$
- Good embedding preserves graph proximity: similar nodes $\rightarrow$ similar vectors

- **Objective Function (Generic):**

- $\mathcal{L} = \sum_{(u,v) \in E} \text{similarity}(\mathbf{z}_u, \mathbf{z}_v) - \sum_{(u,v) \notin E} \text{similarity}(\mathbf{z}_u, \mathbf{z}_v)$
- Maximize similarity for connected nodes, minimize for unconnected nodes



Graph Embeddings



L-Università ta' Malta
Faculty of Information &
Communication Technology

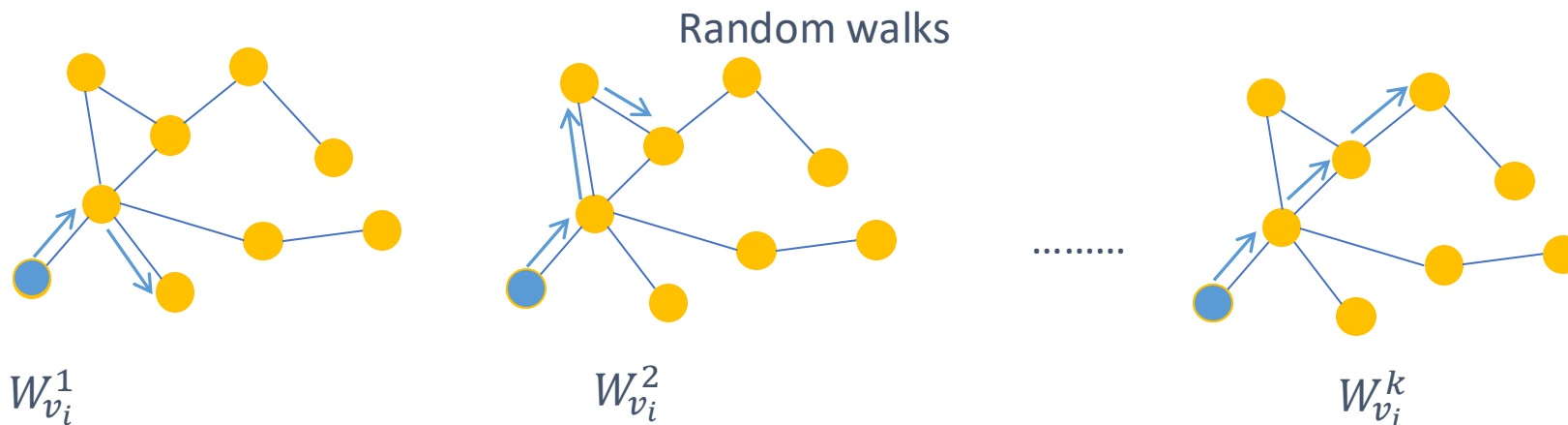
Department
of Artificial
Intelligence

- **Properties of Ideal Embeddings:**

- **Proximity Preservation:** Connected nodes have similar embeddings
- **Community Awareness:** Nodes in same community cluster together
- **Role Equivalence:** Nodes with similar structural roles have similar embeddings
- **Dimensionality Reduction:** Compress high-dimensional graph to manageable vectors

- **Evolution of Embedding Methods:**

- **Random-Walk Based:** DeepWalk, Node2Vec (capture local structure)
- **Neural Approaches:** Graph Autoencoders, Graph Neural Networks (incorporate attributes)



Skip-Gram

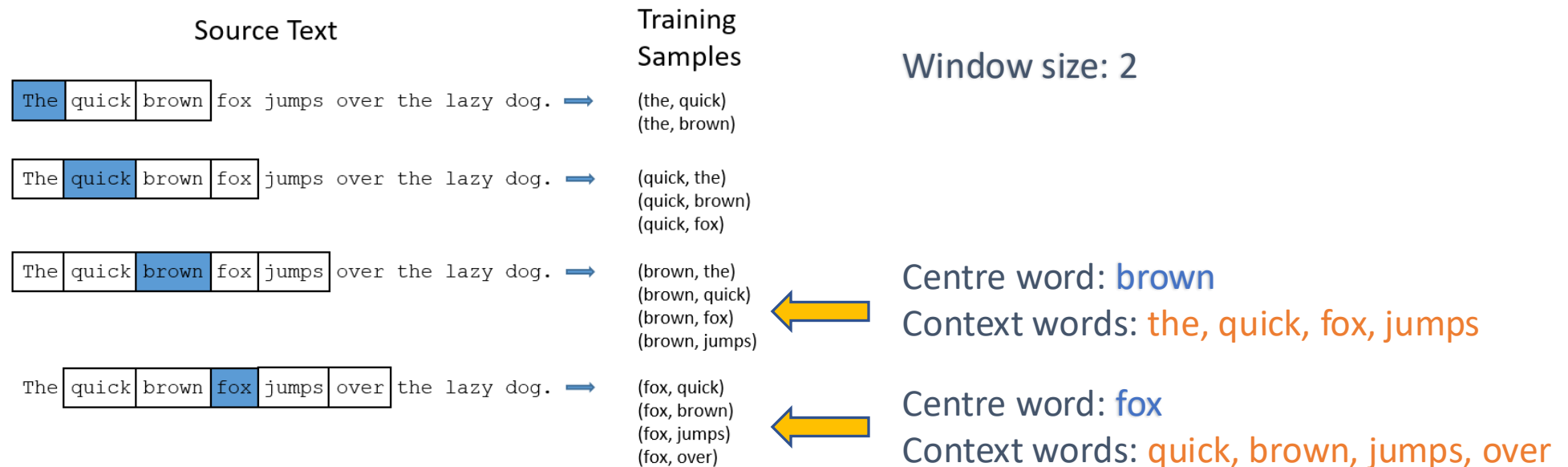


- word2vec is a word embedding technique that represents words as a vector.
- The **Skip-Gram** neural network is used to learn the embeddings of words based on their contexts.
 - words that have a lot of contexts in common result in similar vectors.
- In DeepWalk, rather than train the network to get at the output, the goal is to learn the weights of the hidden layer.

Skip-Gram



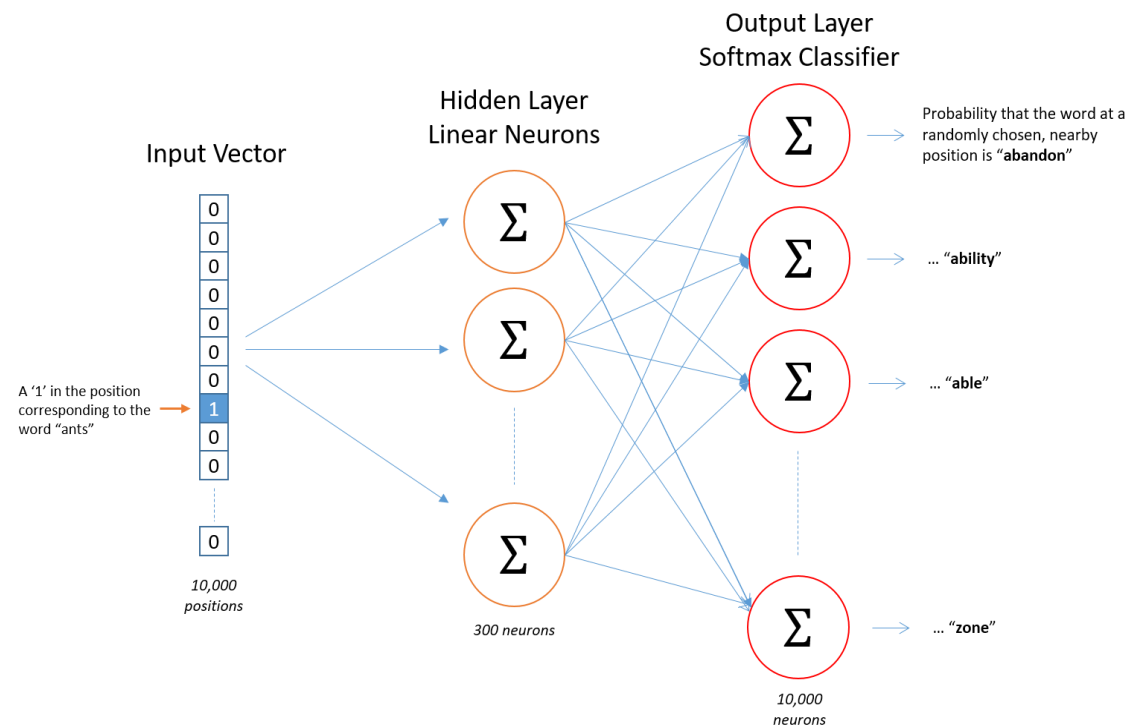
- The model is trained using specific words in a sentence (centre words) and the words surrounding it (context words)
 - context words => defined by some window size e.g. 2, means two words before and two words after
 - the output probabilities are indicative of the likelihood of finding these words nearby



Skip-Gram



- Skip-Gram uses a neural network with a single hidden layer and applies softmax over the output.



<http://mccormickml.com/>

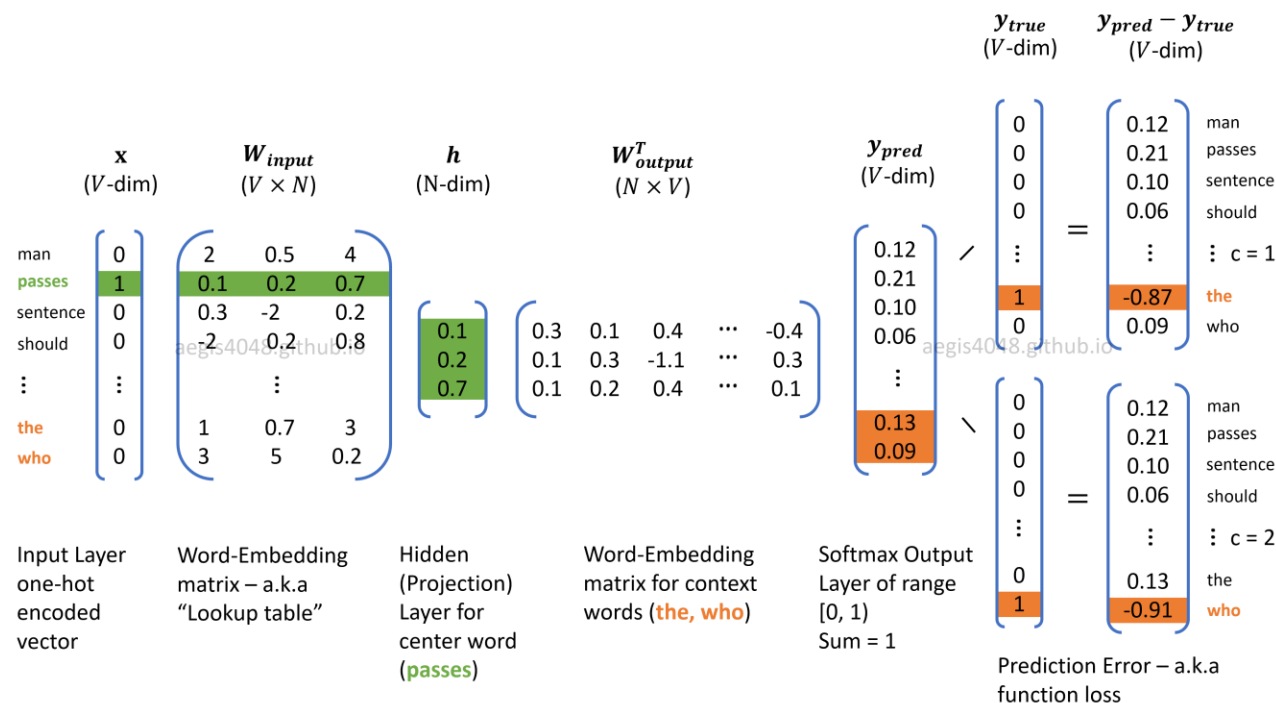
- There is no activation function in the hidden layer.
- When training, the input and output are one-hot encoding vectors representing the input and output words respectively.
- When evaluating the trained network on some input word, the output will be a probability distribution (generated using softmax)

Skip-Gram: Example



“The man who passes the sentence should swing the sword” – Ned Stark

- The input (vocabulary) is this sentence, with just 8 unique words
- Assume a window size of 1: The man **who** **passes** **the** sentence should swing the sword. → (passes, who)
(passes, the)
- Assume a hidden layer with 3 neurons => $W_{input} = 8 \times 3$ and $W_{output}^T = 3 \times 8$



softmax

$$\frac{e^x}{\sum e^x}$$

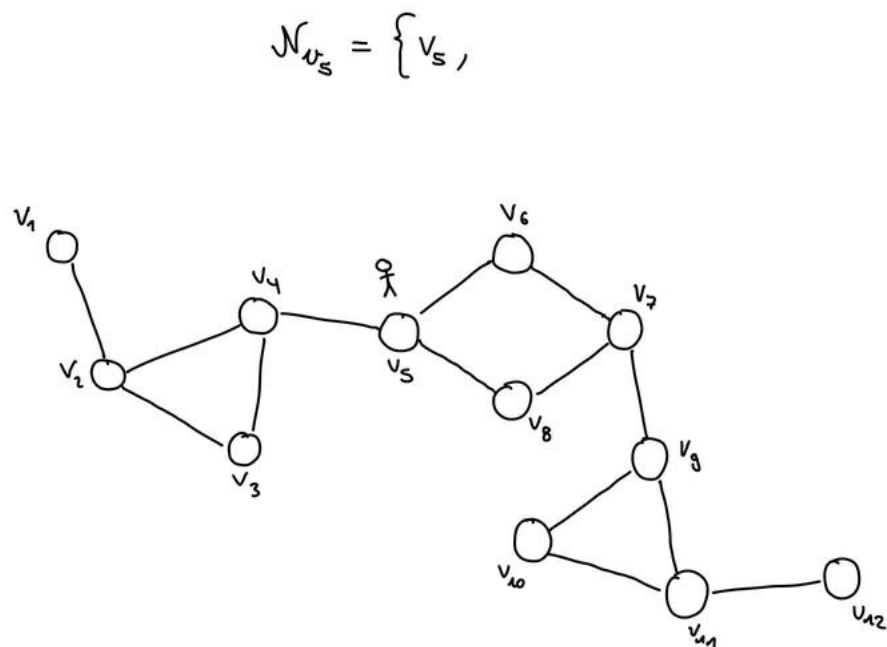
Probability that if a random word is picked nearby “passes”, it is “the” or “who”

Applied to DeepWalk



- The similarity (u, v) is defined as the probability that u and v co-occur on a random walk over a network.
- Random walk embeddings are generated as follows:
 - Estimate the probability of visiting node v on a random walk starting from some node u using some random walk strategy R . For example: using fixed-length, unbiased random walks starting from each node (*Perozzi et al 2013*)
 - Optimise embeddings to encode these random walk statistics, so the similarity between embeddings (based on the dot product) encodes Random Walk similarity.

Applied to DeepWalk



acquired random walk

$$\mathcal{N}_{v_5} = \{v_5, v_8, v_7, v_9, v_{10}, v_9, v_{11}, v_9, v_7\}$$

$$\mathcal{N}_{v_5} = \{v_5, v_8, v_7, v_9, v_{10}, v_9, v_{11}, v_9, v_7\} \quad (v_5, v_8), (v_5, v_7)$$

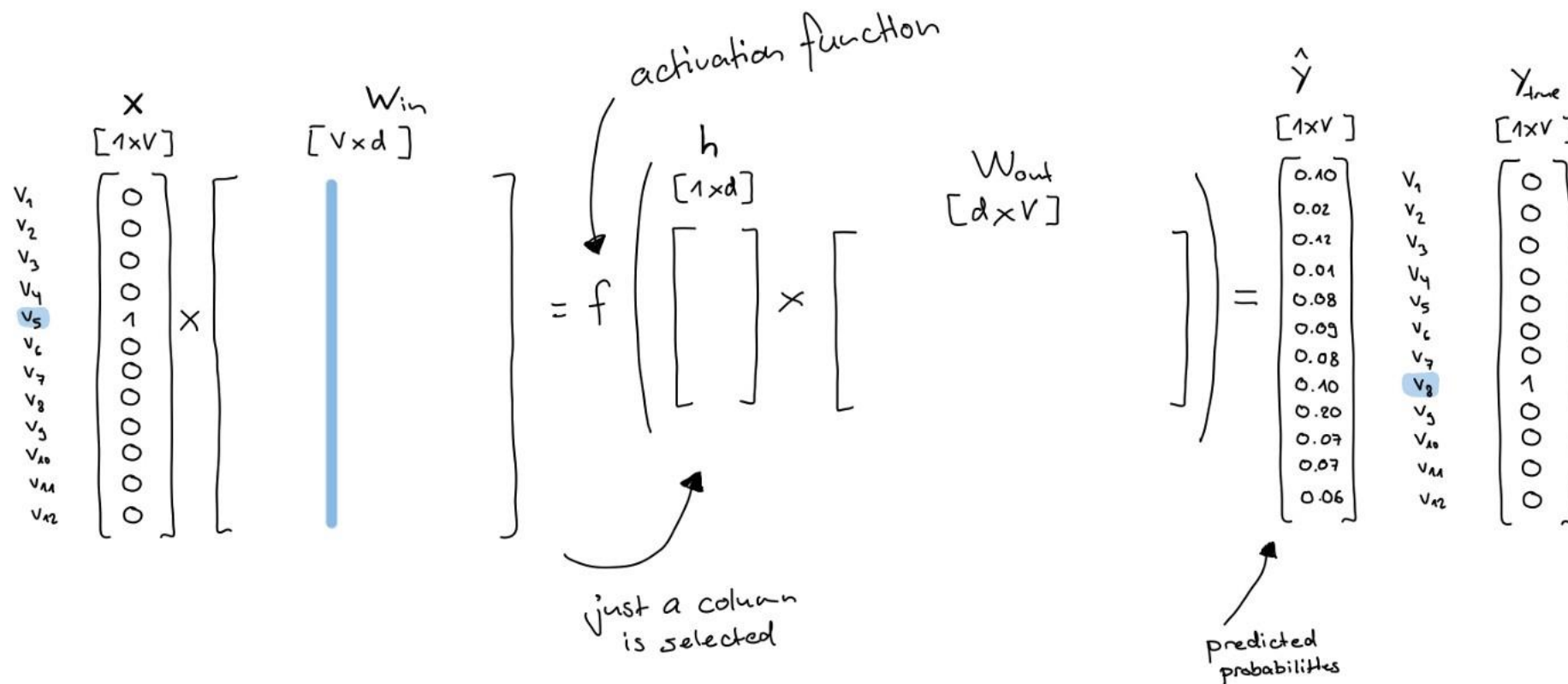
$$\mathcal{N}_{v_5} = \{v_5, v_8, v_7, v_9, v_{10}, v_9, v_{11}, v_9, v_7\} \quad (v_8, v_5), (v_8, v_7), (v_8, v_9)$$

$$\mathcal{N}_{v_5} = \{v_5, v_8, v_7, v_9, v_{10}, v_9, v_{11}, v_9, v_7\} \quad (v_7, v_5), (v_7, v_8), (v_7, v_9), (v_7, v_{10})$$

training samples

window

Applied to DeepWalk



node2vec



- The random walk approach is considered to be rigid and is insensitive to connectivity patterns unique to networks:
 - nodes can be organised based on communities (homophily) they belong to, such as nodes u and s_1
 - node organisation could be based around structural roles of nodes within the network
 - nodes which although far apart in the network, exhibit similar characteristics such as nodes u and s_6

node2Vec: How It Works



- **Intuition:**

- Inspired by Word2Vec from NLP: words appearing in similar contexts have similar meanings
- In graphs: nodes appearing in similar random walk contexts have similar roles/properties

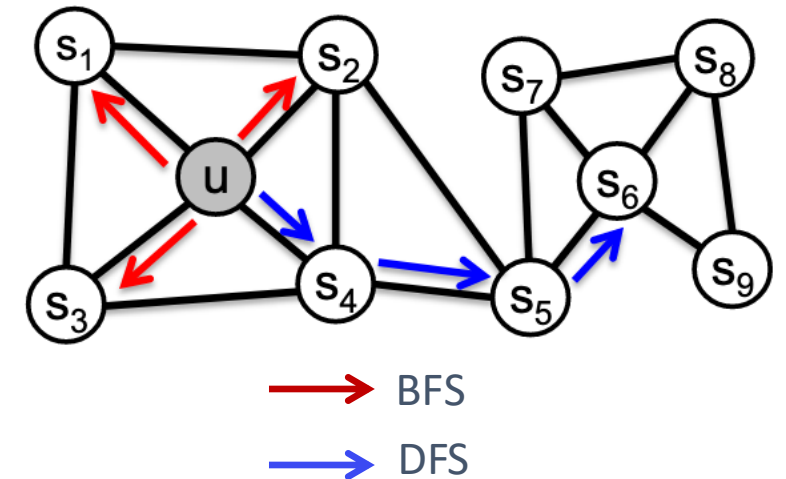
- **Algorithm Steps:**

1. Generate biased random walks from each node
2. Create node contexts from these walks
3. Apply Skip-gram model to learn node representations

node2vec



- node2vec is a semi-supervised algorithm for scalable feature learning in graphs
 - allows for biased random walks using different sampling strategies over a node's neighbourhood
 - learns representations that embed nodes from the same network community by leveraging on breadth first search (BFS)
 - learns representations that embed nodes that correspond closely to structural equivalence by leveraging on depth first search (DFS)
- Walk of length 3 ($N_R(u)$ of size 3)
 - $N_{BFS}(u) = \{s_1, s_2, s_3\} \Rightarrow$ local microscopic view
 - $N_{DFS}(u) = \{s_4, s_5, s_6\} \Rightarrow$ global macroscopic view



node2vec: Scalable Feature Learning for Networks. A. Grover, J. Leskovec. *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

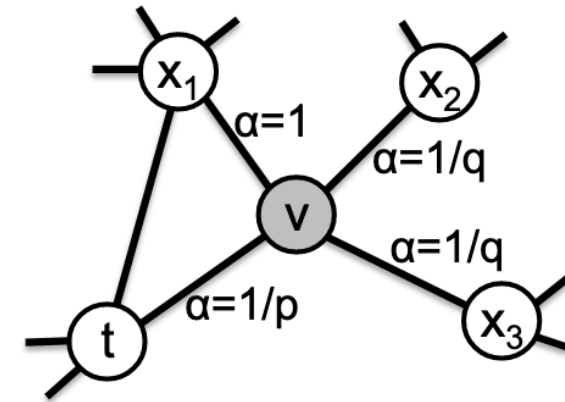
node2vec



- node2vec leverages on a trade-off between BFS and DFS strategies
- bias α_{pq} is used to interpolate between BFS and DFS
- p is the return parameter and q is the in-out parameter

$$\pi_{vx} = \alpha_{pq}(t, x) \cdot w_{vx}$$

transition probability search bias edge weight



where

$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases}$$

d_{tx} = shortest path distance between t and x
 $d_{tx} = \{0, 1, 2\}$
 $d_{tx} = 0 \Rightarrow t$ and x are the same, return to the previous node
 $d_{tx} = 1 \Rightarrow x$ is adjacent to both v and t , encourage BFS
 $d_{tx} = 2 \Rightarrow x$ is not adjacent to either v nor t , encourage DFS

Limitations



- **Limitations** of shallow embedding methods like DeepWalk and node2vec:
 - $O(|V|)$ parameters are needed:
 - No sharing of parameters between nodes
 - Every node has its own unique embedding
 - Inherently “transductive”:
 - Cannot generate embeddings for nodes that are not seen during training
 - Do not incorporate node features:
 - Many graphs have features that we can and should leverage on

Using Embeddings for Anomaly Detection



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

General Approach:

1. Generate node embeddings $\{\mathbf{z}_v \in \mathbb{R}^d: v \in V\}$
2. Apply traditional anomaly detection to embeddings
3. Rank nodes by anomaly score

Detection Methods on Embeddings:

Distance-based:

Compute centroid: $\boldsymbol{\mu} = \frac{1}{|V|} \sum_{v \in V} \mathbf{z}_v$

Score node by distance: $\text{score}(v) = \|\mathbf{z}_v - \boldsymbol{\mu}\|^2$

Anomalies = nodes with highest scores

Density-based (LOF):

Compute Local Outlier Factor for each embedding:

$$\text{LOF}_k(v) = \frac{\sum_{o \in N_k(v)} \frac{\text{lrd}_k(o)}{|N_k(o)|}}{|N_k(v)|} \frac{\text{lrd}_k(v)}{|N_k(v)|}$$

Where $\text{lrd}_k(v)$ is local reachability density and $N_k(v)$ is k-neighborhood
High LOF indicates anomaly (low local density compared to neighbors)

Cluster-based:

Apply clustering (e.g., K-means) to embeddings

Score nodes by distance to nearest cluster center

Or identify small/isolated clusters as anomalous

Using Embeddings for Anomaly Detection: Example



```
import numpy as np
from node2vec import Node2Vec
from sklearn.neighbors import LocalOutlierFactor

# Generate embeddings
n2v = Node2Vec(G, dimensions=64, walk_length=30, num_walks=200, p=1, q=1)
model = n2v.fit(window=10, min_count=1)

# Extract embeddings
embeddings = np.array([model.wv[node] for node in G.nodes()])

# LOF anomaly detection
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
outlier_scores = -lof.fit_predict(embeddings) # -1 for outliers, 1 for inliers

# Map scores back to nodes
node_scores = {node: score for node, score in zip(G.nodes(), lof.negative_outlier_factor_)}
anomalies = [node for node, score in node_scores.items() if score < threshold]
```

Graph Neural Networks: Why We Need Them



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Limitations of Traditional Embeddings:**
 - Transductive Only: Cannot generate embeddings for new, unseen nodes
 - Feature Ignorance: Many methods don't incorporate node/edge attributes
 - Fixed Representations: Cannot adapt to dynamic graphs without retraining
 - Limited Expressiveness: May not capture complex structural patterns
- **GNN Advantages:**
 - Inductive Learning: Can generalize to unseen nodes with learned message-passing
 - Feature Integration: Naturally incorporate node and edge attributes
 - End-to-End Differentiable: Can be trained jointly with downstream tasks
 - Higher Expressiveness: Capture complex, non-linear relationships in graph data
 - Adaptability: Can handle evolving graphs through re-computation
- **Paradigm Shift:**
 - From "embedding lookup tables" to "embedding functions"
 - From "node similarity" to "neighborhood aggregation"
 - From "fixed vectors" to "context-dependent representations"

Graph Neural Networks: The Intuition



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Core Philosophy:**

- Nodes are defined by their neighborhoods
- Features propagate through the graph
- Multi-hop information aggregation captures broader context

- **Message Passing Paradigm:**

1. **Initialize:** Each node starts with its features $\mathbf{h}_v^{(0)} = \mathbf{x}_v$
2. **Aggregate:** Nodes collect messages from neighbors
3. **Update:** Nodes update their representation based on messages
4. **Repeat:** Multiple layers capture k-hop neighborhoods

General Formula:

Aggregate: $\mathbf{a}_v^{(l)} = \text{AGGREGATE}^{(l)}(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\})$

Update: $\mathbf{h}_v^{(l)} = \text{UPDATE}^{(l)}(\mathbf{h}_v^{(l-1)}, \mathbf{a}_v^{(l)})$

After L layers, $\mathbf{h}_v^{(L)}$ contains information from L-hop neighborhood

Variants and Aggregation Strategies:

Mean Aggregator: Average of neighbor features

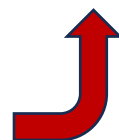
Max Pooling: Element-wise maximum of transformed neighbor features

Attention-based: Weighted average where weights are learned attention coefficients

Analogy to CNNs:

CNNs: Convolve over regular grid of pixels

GNNs: Convolve over irregular neighborhood structures



Graph Convolutional Networks



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Intuition:**

- Generalize convolution operation from regular grids (images) to irregular graph structures
- Each node aggregates and transforms features from its neighborhood

- **Original GCN Layer (Kipf & Welling, 2017):**

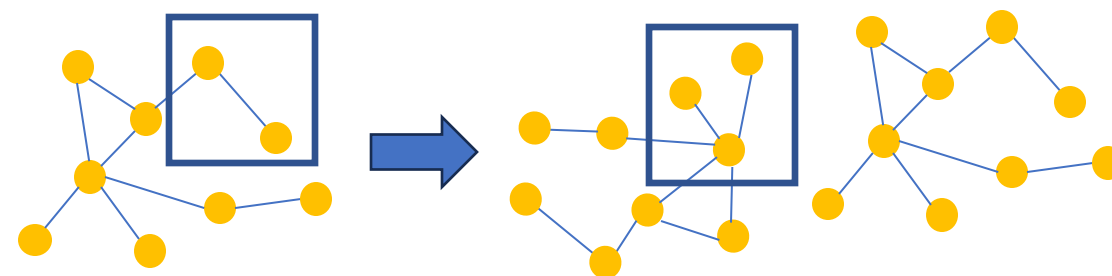
- $\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)})$
- Where:
 - $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ (adjacency matrix with self-loops)
 - $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$ (diagonal matrix with $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$)
 - $\mathbf{H}^{(l)}$ is the matrix of node features at layer l (initially $\mathbf{H}^{(0)} = \mathbf{X}$)
 - $\mathbf{W}^{(l)}$ is the trainable weight matrix for layer l
 - σ is a non-linear activation function (e.g., ReLU)

- **Normalization Term: $\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$**

- Prevents exploding/vanishing gradients
- Accounts for varying neighborhood sizes
- Symmetric normalization preserves eigenvalues

- **From Matrix to Node-Level View:**

- For a single node v , the update is: $\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} \mathbf{W}^{(l)} \mathbf{h}_u^{(l)}\right)$
- Essentially a weighted average of transformed neighbor features



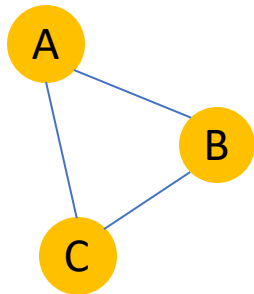
There is no fixed notion of locality or sliding window on the graph

GCN Step-by-Step Example



- **Setup: Simple 3-Node Graph**

- Nodes: A, B, C
- Edges: (A,B), (B,C), (A,C)
- Node Features (1D for simplicity):



Step 1: Prepare Matrices

$$\text{Adjacency Matrix: } \mathbf{A} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

$$\text{Add Self-Loops: } \tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Degree Matrix: } \tilde{\mathbf{D}} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

$$\text{Normalized Adjacency: } \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{3}} \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}$$

$$\text{Feature Matrix: } \mathbf{X} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \text{ (usually based on random values from specific distribution)}$$

$$\text{Weight Matrix (given): } \mathbf{W}^{(0)} = [2]$$

GCN Step-by-Step Example



Step 2: First Layer Forward Pass

$$\mathbf{H}^{(1)} = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} \mathbf{W}^{(0)})$$

$$\mathbf{Z}^{(1)} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} [2] = \begin{bmatrix} \frac{1}{3}(1+2+3) \\ \frac{1}{3}(1+2+3) \\ \frac{1}{3}(1+2+3) \end{bmatrix} [2] = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} [2] = \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}$$

$$\text{Applying ReLU: } \mathbf{H}^{(1)} = \text{ReLU}(\mathbf{Z}^{(1)}) = \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}$$

Step 3: Node-Level Interpretation

For **node A**

Features of neighbors (including self): $\mathbf{x}_A = 1$, $\mathbf{x}_B = 2$, $\mathbf{x}_C = 3$

Normalized aggregation:

$$\frac{1}{\sqrt{3} \cdot \sqrt{3}} \cdot 1 + \frac{1}{\sqrt{3} \cdot \sqrt{3}} \cdot 2 + \frac{1}{\sqrt{3} \cdot \sqrt{3}} \cdot 3 = \frac{1}{3}(1+2+3) = 2$$

Apply weight and activation: $\text{ReLU}(2 \cdot 2) = 4$

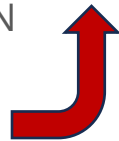
Similar for B and C $\rightarrow$ all nodes have value 4 after first layer

Insights: GCN layer performs weighted neighbourhood averaging. Information propagates across edges. After one layer, each node has information from its 1-hop neighbourhood. After two layers, would have information from 2-hop neighbourhood. In this fully-connected example, all nodes converge to the same representation

Graph Autoencoders for Anomaly Detection



- **Core Concept:**
 - Apply autoencoder architecture to graph-structured data
 - Learn to embed and reconstruct graph properties
 - Use reconstruction error to identify anomalies
- **Architecture Components:**
 - **Encoder:** GNN layers (e.g., GCN) transform node features and structure to embeddings
 - $\mathbf{Z} = \text{GNN}_{\text{enc}}(\mathbf{A}, \mathbf{X})$ where $\mathbf{Z} \in \mathbb{R}^{n \times d}$ is node embeddings
 - **Decoder:**
 - Structure decoder: $\hat{\mathbf{A}} = \sigma(\mathbf{Z}\mathbf{Z}^T)$ (inner product of embeddings)
 - Feature decoder: $\hat{\mathbf{X}} = \text{MLP}(\mathbf{Z})$ or another GNN
- **Loss Functions:**
 - Structure reconstruction loss: $\mathcal{L}_{\text{struct}} = \|\mathbf{A} - \hat{\mathbf{A}}\|_F^2$ or cross-entropy
 - Feature reconstruction loss: $\mathcal{L}_{\text{feat}} = \|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$
 - Combined loss: $\mathcal{L} = \alpha \mathcal{L}_{\text{struct}} + (1 - \alpha) \mathcal{L}_{\text{feat}}$
- **Anomaly Scores:**
 - Node-level structural anomaly: $s_{\text{struct}}(v) = \sum_{u \in V} (\mathbf{A}_{vu} - \hat{\mathbf{A}}_{vu})^2$
 - Node-level attribute anomaly: $s_{\text{feat}}(v) = \|\mathbf{X}_v - \hat{\mathbf{X}}_v\|^2$
 - Combined score: $s(v) = \alpha \cdot s_{\text{struct}}(v) + (1 - \alpha) \cdot s_{\text{feat}}(v)$
- **Algorithm: DOMINANT (Deep Anomaly Detection on Attributed Networks)**
 - Uses GCN encoder for both structure and attribute reconstruction
 - Captures nodes that are anomalous in structure, attributes, or both
 - Optimized for attributed networks with both connection and feature anomalies



Graph Autoencoders for Anomaly Detection



```
class GraphAutoencoder(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()
        # Encoder (GCN layers)
        self.conv1 = GCNConv(input_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, latent_dim)

        # Feature Decoder (MLP)
        self.feats_decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim)
        )

    def encode(self, x, edge_index):
        h = self.conv1(x, edge_index).relu()
        z = self.conv2(h, edge_index)
        return z

    def decode_structure(self, z):
        # Inner product decoder for adjacency
        return torch.sigmoid(torch.matmul(z, z.t()))

    def decode_features(self, z):
        # MLP decoder for node features
        return self.feats_decoder(z)

    def forward(self, x, edge_index):
        z = self.encode(x, edge_index)
        adj_recon = self.decode_structure(z)
        feat_recon = self.decode_features(z)
        return z, adj_recon, feat_recon
```

Graph Autoencoder: Numerical Walkthrough



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

Example Scenario:

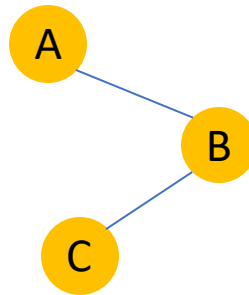
Small graph with 3 nodes: A, B, C

Edges: (A,B), (B,C)

Node features: Features (1D for simplicity):

$$\text{Adjacency matrix: } \mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\text{Feature matrix: } \mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$



Step 1: Encoding

After GCN layers (simplified calculation), get embeddings:

$$\mathbf{z}_A = [0.3, 0.7]$$

$$\mathbf{z}_B = [0.5, 0.5]$$

$$\mathbf{z}_C = [0.7, 0.3]$$

$$\text{Embedding matrix: } \mathbf{Z} = \begin{bmatrix} 0.3 & 0.7 \\ 0.5 & 0.5 \\ 0.7 & 0.3 \end{bmatrix}$$

Graph Autoencoder: Numerical Walkthrough



Step 2: Structure Decoding

Inner product decoder: $\hat{\mathbf{A}} = \sigma(\mathbf{Z}\mathbf{Z}^T)$

Calculation:

$$\mathbf{Z}\mathbf{Z}^T = \begin{bmatrix} 0.3 & 0.7 \\ 0.5 & 0.5 \\ 0.7 & 0.3 \end{bmatrix} \begin{bmatrix} 0.3 & 0.5 & 0.7 \\ 0.7 & 0.5 & 0.3 \end{bmatrix} = \begin{bmatrix} 0.58 & 0.5 & 0.42 \\ 0.5 & 0.5 & 0.5 \\ 0.42 & 0.5 & 0.58 \end{bmatrix}$$

$$\text{Apply sigmoid: } \hat{\mathbf{A}} = \begin{bmatrix} 0.64 & 0.62 & 0.60 \\ 0.62 & 0.62 & 0.62 \\ 0.60 & 0.62 & 0.64 \end{bmatrix}$$

Step 3: Feature Decoding

Apply MLP to each embedding (simplified):

$$\hat{\mathbf{X}}_A = [0.9, 0.1]$$

$$\hat{\mathbf{X}}_B = [0.4, 0.8]$$

$$\hat{\mathbf{X}}_C = [0.8, 0.9]$$

$$\text{Reconstructed feature matrix: } \hat{\mathbf{X}} = \begin{bmatrix} 0.9 & 0.1 \\ 0.4 & 0.8 \\ 0.8 & 0.9 \end{bmatrix}$$

Graph Autoencoder: Numerical Walkthrough



Step 4: Error Calculation

Structure reconstruction error (row-wise MSE):

$$e_{\text{struct}}(A) = \frac{1}{3} [(0 - 0.64)^2 + (1 - 0.62)^2 + (0 - 0.60)^2] = 0.455$$

$$e_{\text{struct}}(B) = \frac{1}{3} [(1 - 0.62)^2 + (0 - 0.62)^2 + (1 - 0.62)^2] = 0.168$$

$$e_{\text{struct}}(C) = \frac{1}{3} [(0 - 0.60)^2 + (1 - 0.62)^2 + (0 - 0.64)^2] = 0.455$$

Feature reconstruction error:

$$e_{\text{feat}}(A) = \frac{1}{2} [(1 - 0.9)^2 + (0 - 0.1)^2] = 0.01$$

$$e_{\text{feat}}(B) = \frac{1}{2} [(0 - 0.4)^2 + (1 - 0.8)^2] = 0.13$$

$$e_{\text{feat}}(C) = \frac{1}{2} [(1 - 0.8)^2 + (1 - 0.9)^2] = 0.025$$

Combined scores ($\alpha = 0.5$):

$$s(A) = 0.5 \cdot 0.455 + 0.5 \cdot 0.01 = 0.2325$$

$$s(B) = 0.5 \cdot 0.168 + 0.5 \cdot 0.13 = 0.149$$

$$s(C) = 0.5 \cdot 0.455 + 0.5 \cdot 0.025 = 0.24$$

Step 5: Anomaly Detection

Sort nodes by score: C (0.24) > A (0.2325) > B (0.149)

If threshold = 0.2, nodes A and C would be flagged as anomalies

C has highest score due to combination of structural and feature deviation

Case Study: Fraud Detection in Bitcoin Transactions



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Problem Statement:**

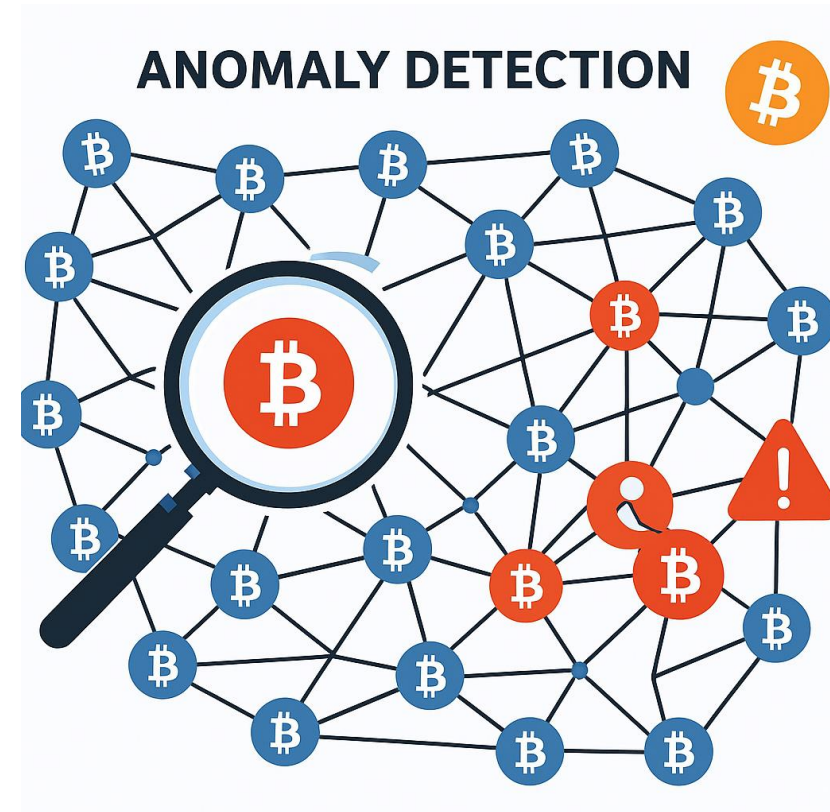
- Identify illicit transactions in Bitcoin blockchain
- Transactions form a complex network (money flow)
- Challenge: Disguised patterns, evolving techniques

- **Dataset: Elliptic Data Set**

- 203,769 Bitcoin transactions (nodes)
- 234,355 payment flows (directed edges)
- 166 node features (transaction characteristics)
- Classes: 4,545 illicit, 42,019 licit, rest unknown

- **Graph Characteristics:**

- Directed temporal graph (transactions over time)
- Node features include:
 - Local features (transaction amount, time step)
 - Aggregated features (in/out degree, transaction fee)
- 49 connected components representing different timeframes



Case Study: Fraud Detection in Bitcoin Transactions



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **GNN-Based Approach:**

- **Model:** Graph Convolutional Network with attention
- **Architecture:**
 - 2 GCN layers: $166 \rightarrow 128 \rightarrow 64$ features
 - Attention mechanism to focus on suspicious patterns
 - Final classifier: $64 \rightarrow 32 \rightarrow 2$ (licit/illicit)
- **Training:**
 - Semi-supervised learning with labelled transactions
 - Loss function: Cross-entropy + graph regularization
 - Training on 70% of labelled data

Results:

AUC: 0.96 (significantly better than non-graph baselines at 0.85)

Precision: 0.92, Recall: 0.89 for illicit class

Key patterns identified:

Money laundering cycles (funds returning to origin)

Rapid fan-out transactions (splitting funds)

Mixer services (combining multiple sources)

Case Study: Fraud Detection in Bitcoin Transactions



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

- **Explainability:**

- Attention weights reveal influential neighbours
- Feature importance highlights suspicious characteristics
- Subgraph patterns help investigators understand techniques

- **Business Impact:**

- 87% reduction in false positives vs. rule-based systems
- 95% of large fraudulent transactions identified before completion
- \$1.2M in potential fraud prevented in 6-month pilot

Synthesis and Future Directions



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

• Key Takeaways from Our Journey:

- **Context is Crucial:** Temporal patterns and structural relationships provide essential detection signals
- **LSTM Power:** Cell state and gates enable learning of complex sequential dependencies
 - Perfect for time series with long-range patterns and variable contexts
- **GNN Revolution:** Message passing learns from both structure and attributes
 - Captures complex relationships impossible with traditional methods
- **Autoencoders:** Unsupervised learning of normal patterns enables anomaly detection without labeled anomalies
 - Works for both sequences (LSTM-AE) and graphs (GAE)
- **Complementarity:** Time series and graph techniques offer different perspectives on the same data

Synthesis and Future Directions



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

• Practical Implementation Guidance:

- Start simple: Statistical methods provide baselines
- Choose algorithms based on data characteristics and anomaly types
- Ensemble methods often outperform single algorithms
- Hardware considerations: GPUs crucial for large graphs and deep LSTMs
- Evaluation requires domain-specific metrics and human validation

• Resources for Continued Learning:

- Libraries: PyTorch Geometric, DGL, PyG-Temporal, TensorFlow GNN
- Datasets: NAB (time series), DARPA (network intrusion), Elliptic (Bitcoin)
- Research Groups: Stanford GraphML, Montreal AI, NYU Graph Mining